Code Explanation

Testing the ETL Module

The provided code is a test suite for an ETL (Extract, Transform, Load) module written in Python using PySpark. The test suite covers various functions within the ETL module to ensure they work as expected.

Overview of the Code

This test suite includes tests for the ETL module's functions, such as getting a Spark session, saving a DataFrame, and running a pipeline. The tests utilize Pytest and Mocking to isolate dependencies and ensure the ETL module's functionality.

Step 1: Importing Necessary Modules and Defining Test Functions
The test suite begins by importing necessary modules, including Pytest, Mocking from unittest, and various PySpark components. It also imports the functions to be tested from the ETL module.

```
import pytest
from unittest.mock import patch
from pyspark.sql import DataFrame, SparkSession
from src.config import DataSource, PipelineConfig
from src.etl import get_spark_session, run_pipeline, save_dataframe
```

Step 2: Testing the get_spark_session Function
The first test function, `test_get_spark_session`, checks if the `get_spark_session` function returns a valid SparkSession with the correct application name.

```
def test_get_spark_session():
    spark = get_spark_session("Test Session")
    assert isinstance(spark, SparkSession)
    assert spark.conf.get("spark.app.name") == "Test Session"
```

Step 3: Testing the save_dataframe Function
The `test_save_dataframe` function tests the `save_dataframe` function by creating a sample DataFrame, saving it to a temporary location, and then reading it back to verify the data was written correctly.

```
def test_save_dataframe(spark, tmp_path):
    df = spark.createDataFrame([
        (1, "test1"),
        (2, "test2")
    ], ["id", "name"])
    output_path = str(tmp_path / "test_output")
    save_dataframe(df, output_path, format="parquet", mode="overwrit
e")
    read_df = spark.read.parquet(output_path)
    assert read_df.count() == 2
    assert "id" in read_df.columns
    assert "name" in read_df.columns
```

Step 4: Testing the run_pipeline Function with Mocking
The `test_run_pipeline` function is a test for the `run_pipeline` function. It uses Mocking to isolate dependencies, such as `extract_data`, `clean_and_transform_data`, `enrich_sales_data`, `add_sales_metrics`, and `save_dataframe`, allowing the test to focus on the `run_pipeline` function's logic.

```
@patch("src.etl.save_dataframe")
@patch("src.etl.add_sales_metrics")
@patch("src.etl.enrich_sales_data")
@patch("src.etl.clean_and_transform_data")
@patch("src.etl.extract_data")
def test_run_pipeline
```